

Experiment No. – 5

Aim –

- (A.) A website maintains records of users who have registered on the platform and the status of confirmation requests sent to them. A confirmation request can either be successfully completed or may expire without completion.

For each registered user, calculate the confirmation rate, which is defined as:

$$\text{Confirmation Rate} = \frac{\text{Number of Successful Confirmations}}{\text{Total Confirmation Requests}}$$

- (B.) A ride-booking company maintains information about trip requests and the users involved in those trips. Some users may be restricted from using the platform, while others are active participants. Each trip can either be completed successfully or cancelled by one of the participants.

For each day between **2013-10-01** and **2013-10-03**, determine the proportion of canceled trips among all valid trips. A trip should be considered valid only when both the customer and the driver associated with that trip are active users. The final result should display the date and the calculated cancellation rate rounded to two decimal places

Tools Required

- Postgres
- PgAdmin4

Queries:

A.

```
CREATE TABLE Signups (  
    user_id INT PRIMARY KEY,  
    time_stamp TIMESTAMP  
);  
  
CREATE TABLE Confirmations (  
    user_id INT,  
    time_stamp TIMESTAMP,  
    action VARCHAR(20),  
    PRIMARY KEY(user_id, time_stamp),  
    FOREIGN KEY(user_id)
```

```
REFERENCES Signups(user_id)
);
```

```
INSERT INTO Signups
VALUES
(3,'2020-03-21 10:16:13'),
(7,'2020-01-04 13:57:59'),
(2,'2020-07-29 23:09:44'),
(6,'2020-12-09 10:39:37');
```

```
INSERT INTO Confirmations
VALUES
(3,'2021-01-06 03:30:46','timeout'),
(3,'2021-07-14 14:00:00','timeout'),
(7,'2021-06-12 11:57:29','confirmed'),
(7,'2021-06-13 12:58:28','confirmed'),
(7,'2021-06-14 13:59:27','confirmed'),
(2,'2021-01-22 00:00:00','confirmed'),
(2,'2021-02-28 23:59:59','timeout');
```

```
select s.user_id , ROUND(AVG(
Case
    WHEN c.action = 'confirmed' then 1
    else 0
END
) ,2)
from Signups as s
left Join Confirmations as c
ON c.user_id = s.user_id
Group by s.user_id;
```

B.

```
CREATE TABLE Users (  
    users_id INT PRIMARY KEY,  
    banned VARCHAR(3),  
    role VARCHAR(20)  
);
```

```
CREATE TABLE Trips(  
    id INT PRIMARY KEY,  
    client_id INT,  
    driver_id INT,  
    city_id INT,  
    status VARCHAR(30),  
    request_at DATE );
```

INSERT INTO Users VALUES

```
(1,'No','client'),  
(2,'Yes','client'),  
(3,'No','client'),  
(4,'No','client'),  
(10,'No','driver'),  
(11,'No','driver'),  
(12,'No','driver'),  
(13,'No','driver');
```

INSERT INTO Trips VALUES

```
(1,1,10,1,'completed','2013-10-01'),  
(2,2,11,1,'cancelled_by_driver','2013-10-01'),  
(3,3,12,6,'completed','2013-10-01'),  
(4,4,13,6,'cancelled_by_client','2013-10-01'),  
(5,1,10,1,'completed','2013-10-02'),  
(6,2,11,6,'completed','2013-10-02'),  
(7,3,12,6,'completed','2013-10-02'),  
(8,2,12,12,'completed','2013-10-03'),  
(9,3,10,12,'completed','2013-10-03'),  
(10,4,13,12,'cancelled_by_driver','2013-10-03');
```

```

SELECT t.request_at AS Day,
       ROUND(AVG(
CASE
    WHEN t.status IN ('cancelled_by_driver','cancelled_by_client') THEN 1
    ELSE 0
END
       ), 2
) AS Cancellation_Rate
FROM Trips AS t
JOIN Users AS c
ON t.client_id = c.users_id
JOIN Users AS d
ON t.driver_id = d.users_id
WHERE c.banned = 'No'
AND d.banned = 'No'
AND t.request_at BETWEEN '2013-10-01' AND '2013-10-03'
GROUP BY t.request_at;

```

Outputs:

	user_id [PK] integer 	round numeric 
1	2	0.50
2	6	0.00
3	3	0.00
4	7	1.00

	day date 	cancellation_rate numeric 
1	2013-10-01	0.33
2	2013-10-02	0.00
3	2013-10-03	0.50

Learning Outcome

After performing these SQL queries, the following learning outcomes were achieved:

1. **Understanding table creation and relationships:** Learned to create tables using **CREATE TABLE**, define **primary keys** and **foreign keys**, and establish relationships between tables.
2. **Using SQL joins:** Learned how to use **LEFT JOIN** to combine data from Signups and Confirmations, including users who have no confirmation records.
3. **Applying conditional logic:** Used the **CASE** statement to convert conditions such as **confirmed/completed** into numerical values (1 and 0) for calculations.
4. **Performing aggregate calculations:** Learned to use functions like **AVG()** and **ROUND()** to calculate the confirmation rate and cancellation rate accurately.